■■ DeepGuard — Digital Forensics System

College Project Presentation, Responsibility Division & Complete Viva Guide

Project Members: Varsha • Poonam • Anushree • Krushi • Sushti | **Final Evaluation 2026**

PART 1: PROJECT OVERVIEW & WORKING WORKFLOW

Project Title: DeepGuard — Explainable CNN-Based Deepfake & Synthetic Media Detection System.
Core Problem: Modern generative models (Face-Swap GANs, Latent Diffusion) make synthetic identities imperceptible. Most existing tools are opaque 'black boxes' that output uncalibrated percentages without explaining where or why tampering occurred. DeepGuard solves this by coupling pre-inference quality diagnostics with an EfficientNet-B0 CNN, calibrated dual-threshold logic, Grad-CAM visual explainability, and a live WebRTC camera studio.

End-to-End System Workflow (Step-by-Step):

1. **User Input:** Upload image (.jpg, .png, .webp), capture live camera photo (WebRTC), or upload video (.mp4, .mov).
2. **Frontend Validation:** Enforces file size limits (15MB Image / 100MB Video) and routes requests via React 18 & Vite.
3. **Backend Routing:** FastAPI decodes raw bytes into OpenCV BGR numpy arrays and dispatches to processing pipelines.
4. **Pre-Inference Quality Audit:** Computes Laplacian Variance for motion blur (threshold 50.0), contrast standard deviation (threshold 20.0), and isolates faces with Haar Cascades + 25% boundary margin padding.
5. **PyTorch CNN Inference:** Cropped faces/frames are normalized (224x224 RGB) and passed to EfficientNet-B0 in torch.no_grad() mode.
6. **Calibrated Decision Engine:** Raw score ≤ 0.35 → **Likely REAL**; Raw score ≥ 0.55 → **Likely FAKE**; between 0.35 and 0.55 → **INCONCLUSIVE**.
7. **Video Temporal Aggregation:** Samples keyframes at 1 FPS; applies top-k pooling (70% max frame + 30% mean) to catch short swaps.
8. **Visual Explainability:** Computes Grad-CAM attention heatmaps on layer features[8] and provides 5-zone anatomical risk breakdown.
9. **User Delivery & Reporting:** Displays interactive split slider, clickable video timeline, and exportable PDF/JSON audit reports.

Layer / Stage	Technology Used	Forensic Function
User Interface	React 18 + Three.js + Tailwind	Interactive 3D visualizer, live camera, split slider
Backend API	FastAPI (Python 3.11) + Uvicorn	Async routes, video background job polling, validation
Preprocessing	OpenCV + Haar Cascades	Blur variance, contrast std dev, 25% margin padding
CNN Model	PyTorch (EfficientNet-B0)	5.3M parameters, compound scaling, <50ms CPU inference
Explainability	Grad-CAM + 5-Zone Analysis	Gradient class activation maps on layer features[8]

PART 2: FIVE-MEMBER RESPONSIBILITY DIVISION & PRESENTATION SCRIPTS

1. Varsha — CNN Architecture, Model Inference & Threshold Calibration

Core Responsibility: EfficientNet-B0 architecture, PyTorch checkpoints, forward pass execution, decision boundaries, and accuracy metrics.
Key Source Files: backend/app/services/model_loader.py, backend/app/core/config.py, weights/facial_efficientnet_b0.pt
What Varsha Must Demonstrate: Explain compound scaling; show how the model evaluates facial tensors; explain the honest dual-threshold rule (≤0.35, ≥0.55).

Varsha's Presentation Script (1.5 Minutes):
"Respected teachers and examiners, good morning. My name is Varsha, and I am responsible for the Machine Learning and Model Architecture in DeepGuard. For our detection engine, we implemented EfficientNet-B0 using PyTorch. Many previous projects used heavy networks like ResNet-50 or VGG-16 with over 25 million parameters that run very slowly on standard hardware. In contrast, EfficientNet-B0 uses compound scaling—balancing depth, width, and resolution simultaneously—giving us high forensic accuracy with only 5.3 million parameters and CPU inference latency under 50 milliseconds.
We established two specialized detection models: one for Facial Manipulation (face swaps and reenactments) and one for AI-Generated Synthetic Images. Crucially, we rejected the naive 0.5 binary cutoff. We implemented an Honest Dual-Threshold Decision Rule: scores 0.35 or lower are classified as Likely Real, scores 0.55 or higher as Likely Fake, and the middle zone between 0.35 and 0.55 is flagged as Inconclusive. I now invite Poonam to explain how our users interact with this model through our frontend interface."

2. Poonam — Frontend UI/UX, 3D Hero Visualizer & Live Camera Studio

Core Responsibility: React 18 components, Three.js 3D face mesh, WebRTC live camera capture, theme switching, responsive design, and verdict displays.
Key Source Files: frontend/src/App.jsx, frontend/src/pages/Analyze.jsx, frontend/src/components/LiveCameraCapture.jsx, frontend/src/components/Hero3D.jsx
What Poonam Must Demonstrate: Live camera photo capture and track cleanup; front/rear flip; 3D visualizer pause optimization; theme toggle.

Poonam's Presentation Script (1.5 Minutes):
"Thank you, Varsha. Respected examiners, my name is Poonam, and I designed and implemented the Frontend and User Experience for DeepGuard. Our frontend is built using React 18, Vite, and Tailwind CSS. On our landing page, we integrated a Three.js 3D Cybernetic Face Mesh that reacts to mouse movements. To prevent CPU lag, it automatically pauses rendering when scrolled out of view or when switching tabs.
In our workspace, we provide three intuitive input methods: Upload Image, Upload Video, and our dedicated Live Camera Studio. The camera module requests user permission, allows switching between front and rear cameras, displays an alignment reticle, and lets the user capture, preview, or retake snapshots. When the camera view closes, all hardware stream tracks are immediately stopped to prevent hardware locks. Once analysis completes, our UI presents the verdict prominently with calibrated confidence bars, anatomical risk cards, and sample quick-test cards. I will now hand over to Anushree to explain the backend architecture."

3. Anushree — Backend Architecture, FastAPI Endpoints & Asynchronous Video Jobs

Core Responsibility: FastAPI application routing, file validation, security sanitization, background job polling, single-port production SPA hosting, and PDF export.
Key Source Files: backend/app/main.py, backend/app/api/routes_analyze.py, backend/app/api/routes_jobs.py, backend/app/services/report_generator.py
What Anushree Must Demonstrate: Swagger API docs at /docs; image analysis route; video background job status polling; temporary file cleanup.

Anushree's Presentation Script (1.5 Minutes):
"Thank you, Poonam. Respected teachers, my name is Anushree, and I built the Backend and API infrastructure using FastAPI. Our backend handles input validation, security, and orchestrates the inference pipeline. When a user submits an image or video, our API validates file headers, formats, and size constraints—limiting images to 15MB and videos to 100MB. For images, we process requests synchronously at /api/v1/analyze/image, returning predictions and heatmap data URIs in under a second.
For videos, processing multiple frames takes several seconds. A synchronous request would freeze the client connection or trigger a browser gateway timeout. Therefore, I built an Asynchronous Background Job System at /api/v1/analyze/video. The server immediately returns a unique Job ID, processes frames in a background worker, updates progress percentage in memory, and allows Poonam's frontend to poll the job status smoothly until completion. I also implemented automated cleanup to delete temporary video files immediately after analysis. I now pass the presentation to Krushi to explain dataset preprocessing and input quality."

4. Krushi — Datasets, Face Extraction, 25% Margin Padding & Media Quality

Core Responsibility: FaceForensics++ / Celeb-DF dataset principles, Haar Cascade face isolation, Non-Maximum Suppression, 25% boundary expansion, blur & contrast checks, 1-FPS video frame extraction.
Key Source Files: backend/app/services/face_extractor.py, backend/app/services/video_processor.py, frontend/src/components/QualityMetrics.jsx
What Krushi Must Demonstrate: Why 25% padding is necessary for face-swap boundary seams; Laplacian blur calculation; lighting contrast checks.

Krushi's Presentation Script (1.5 Minutes):
"Thank you, Anushree. Respected examiners, my name is Krushi, and I am responsible for Datasets, Preprocessing, and Pre-Inference Media Quality Checks. Our project is designed around principles learned from benchmark deepfake datasets like FaceForensics++ and Celeb-DF. A major flaw in basic deepfake detectors is that they crop tightly around the nose and eyes. However, face-swapping algorithms paste a synthesized mask onto a target head, leaving artifacts along the jawline, chin, and hairline. To solve this, I implemented an automatic 25% Forensic Margin Padding that expands detected bounding boxes to capture these crucial blending boundaries.
We also added Pre-Inference Quality Checks: 1. We compute the Variance of the Laplacian to detect motion blur (threshold 50.0). 2. We compute pixel intensity standard deviation to detect poor lighting (threshold 20.0). 3. We verify resolution meets our 224x224 pixel neural receptive field. For videos, rather than reading every single frame which would overwhelm memory, our pipeline samples frames uniformly at 1 frame per second. I now invite Sushti to discuss testing, explainability, and our results."

5. Sushti — Explainability (Grad-CAM), 5-Zone Breakdown, Testing & Future Scope

Core Responsibility: Grad-CAM gradient attention maps on layer features[8], 5-zone anatomical risk calculation, before/after split slider integration, integration testing, limitations, and future scope.
Key Source Files: backend/app/services/model_loader.py, frontend/src/components/HeatmapViewer.jsx, frontend/src/components/VideoTimeline.jsx, DEMO_GUIDE.md
What Sushti Must Demonstrate: Wipe before/after split slider; explain why Grad-CAM is an explainability tool and not legal proof; explain top-k video pooling.

Sushti's Presentation Script (1.5 Minutes):

"Thank you, Krushi. Respected teachers, my name is Sushti, and my responsibility encompasses Explainability, Results Interpretation, System Testing, and Project Limitations. In digital forensics, a prediction without explanation cannot be trusted. To make DeepGuard transparent, we implemented Grad-CAM (Gradient-weighted Class Activation Mapping). By taking the gradients of the manipulation class with respect to the final convolutional layer of EfficientNet, we produce an activation heatmap highlighting the exact pixels that triggered the model's verdict.

In our frontend, Poonam and I integrated an Interactive Before/After Split Slider, allowing investigators to wipe smoothly between the original photo and the neural heatmap. We also break down risk across 5 Anatomical Zones: Forehead, Eyes, Nose, Mouth, and Jawline, answering the critical question: 'Which part of the face was manipulated?' During testing, our system demonstrated reliable separation on benchmark test items, while honestly flagging poor lighting and motion blur as Inconclusive rather than making false claims. In the future, we plan to incorporate audio-visual synchronization to detect voice clones and lip-sync mismatch. Thank you, and our team is now ready for your questions."

PART 3: COMPLETE ROLE-WISE VIVA QUESTIONS & ANSWERS (60 QUESTIONS)

Each member has 12 dedicated questions. Answers are written in short, natural spoken English for instant viva recall.

SECTION A: VIVA QUESTIONS FOR VARSHA (Model, CNN & Metrics)

Q1. What is a CNN, and why is it used for deepfake detection instead of traditional ML?

Answer: A CNN (Convolutional Neural Network) automatically extracts spatial feature hierarchies from images using learnable sliding kernels. Traditional ML (like SVM or Random Forest) requires manual feature extraction. CNNs detect micro-level pixel noise, blend seams, and texture inconsistencies directly from raw image matrices.

Q2. Which CNN architecture does DeepGuard use, and why?

Answer: We use EfficientNet-B0. It has only 5.3 million parameters compared to ResNet-50's 25.6M. EfficientNet uses compound scaling—scaling depth, width, and resolution simultaneously—giving high forensic accuracy with under 50ms inference latency on standard CPUs.

Q3. What do the convolution, activation, and pooling layers do in our model?

Answer: Convolution layers extract spatial features like edges, skin pores, and blend seams. Activation functions (Swish/ReLU) introduce non-linearity so the network learns complex boundaries. Pooling layers downsample feature maps to reduce computation and provide translation invariance.

Q4. What is the difference between training a model and inference?

Answer: Training is the heavy offline process of feeding thousands of labelled real and fake images, calculating loss, and updating weights via backpropagation. Inference is simply running our saved .pt weight checkpoint with torch.no_grad() to classify new uploaded images in milliseconds.

Q5. What is Transfer Learning, and did our project use it?

Answer: Transfer learning means taking a model pre-trained on a massive general dataset (ImageNet with 1.4M images) and fine-tuning its classification head on deepfake datasets. It saves weeks of GPU compute and prevents overfitting on smaller datasets.

Q6. What loss function and optimizer are standard for deepfake binary classification?

Answer: We use Binary Cross-Entropy Loss (BCELoss) because deepfake detection is a two-class problem (Real vs Manipulated). The optimizer is Adam (Adaptive Moment Estimation) with a learning rate of 1e-4 because it dynamically adapts learning rates for each parameter.

Q7. What are Epochs, Batch Size, and Learning Rate?

Answer: An Epoch is one complete forward and backward pass of the entire training dataset. Batch Size is the number of samples processed before weights are updated (e.g., 32 or 64). Learning Rate is the step size taken towards the minimum of the loss function.

Q8. What is Overfitting, and how can it be prevented?

Answer: Overfitting occurs when a model memorizes specific training faces instead of general manipulation artifacts, causing high training accuracy but poor real-world testing. It is prevented using Dropout (0.2), L2 weight decay, data augmentation, and early stopping.

Q9. What is Data Leakage in deepfake research?

Answer: Data leakage occurs when frames from the same video or identity appear in both training and test sets. The model simply memorizes the person's identity instead of detecting deepfakes, producing falsely inflated accuracy. It is prevented by splitting datasets strictly at the video/subject level.

Q10. Explain the Honest Dual-Threshold Decision Rule.

Answer: Standard binary classifiers use a naive 0.5 cutoff. DeepGuard uses two calibrated thresholds: score <= 0.35 is 'Likely Real', score >= 0.55 is 'Likely Fake', and the middle zone (0.35 to 0.55) is flagged as 'Inconclusive'. This prevents borderline, compressed images from false high-confidence labels.

Q11. Explain the difference between Precision and Recall.

Answer: Precision measures how many flagged fakes were actually fake (high precision means few innocent photos are falsely accused). Recall measures how many actual fakes in the dataset were caught (high recall means few deepfakes slip through unnoticed).

Q12. What is an ROC-AUC curve?

Answer: Receiver Operating Characteristic (ROC) plots the True Positive Rate against the False Positive Rate across all thresholds. AUC (Area Under the Curve) measures overall separability. An AUC of 1.0 represents a perfect classifier; 0.5 is random guessing.

■ SECTION B: VIVA QUESTIONS FOR POONAM (Frontend, UI/UX & Live Camera)

Q1. What frontend framework and libraries were chosen, and why?

Answer: We used React 18 with Vite and Tailwind CSS. React provides component-based state management; Vite provides lightning-fast build times; and Tailwind CSS enables a clean, responsive digital forensics design with dark and light theme support.

Q2. How is the Live Camera feature implemented in the browser?

Answer: It uses the HTML5 WebRTC API via navigator.mediaDevices.getUserMedia with facingMode constraints. The live stream is rendered in an HTML5 video element with an alignment reticle overlay.

Q3. How do you prevent camera hardware locks or memory leaks on Windows?

Answer: In LiveCameraCapture.jsx, we store the media stream reference. Inside the React useEffect cleanup return function, and when closing or retaking photos, we explicitly iterate over all tracks and call track.stop(). This immediately releases the webcam hardware.

Q4. Why is the selfie camera mirrored in preview but not inverted during backend analysis?

Answer: Users expect a front-facing selfie camera to act like a physical mirror. We use CSS scaleX(-1) on the video element for comfortable framing. When capturing, we draw to a canvas context, convert it to a JPEG Blob, and send standard orientation to the neural network.

Q5. Can a live camera photo be assumed authentic just because it came from a camera?

Answer: No! A user could point their webcam at an iPad screen displaying a deepfake or hold up a printed synthetic photo. DeepGuard evaluates pixel arrays directly and displays an advisory notice that camera origin does not guarantee authenticity.

Q6. How does the 3D Hero visualizer work, and does it slow down low-end PCs?

Answer: It is built using Three.js to render an interactive wireframe face. To guarantee smooth performance, we use IntersectionObserver to halt the animation loop whenever the user scrolls down, and we pause rendering when the browser tab is hidden.

Q7. How does the interactive Split Comparison Slider work in HeatmapViewer.jsx?

Answer: It renders the original image and the Grad-CAM heatmap overlay. We track mouse/touch drag coordinates across the container width and apply a dynamic CSS clip-path: polygon(...) to reveal the heatmap beneath the original photo in real time.

Q8. How does the frontend handle state when a user switches between Image and Video modes?

Answer: We use React useState hooks in Analyze.jsx. When mode or file type changes, handleReset() clears preview URLs using URL.revokeObjectURL() to prevent browser memory leaks, resets progress counters, and clears previous verdict state.

Q9. How is the theme toggle (Dark / Light mode) persisted?

Answer: We toggle the 'dark' class on the root document.documentElement element and persist the user's preference in browser localStorage. When the user refreshes or reopens the website, their preferred theme is loaded immediately.

Q10. What accessibility (a11y) considerations were implemented?

Answer: We included keyboard-accessible dropzones, distinct color contrasts (avoiding red/green reliance alone by adding explicit text badges and icons), visible focus rings, and respect for prefers-reduced-motion.

Q11. How is file upload drag-and-drop handled?

Answer: We attach onDragOver (with e.preventDefault()) and onDrop event handlers to the dropzone container in Analyze.jsx. When dropped, e.dataTransfer.files[0] is extracted and passed to handleFileChange().

Q12. What does the VerdictCard display when inference completes?

Answer: It displays the verdict badge (Likely REAL / Likely FAKE / INCONCLUSIVE), calibrated confidence percentage, 5-zone anatomical risk cards, media resolution, and an option to report incorrect results.

■■ SECTION C: VIVA QUESTIONS FOR ANUSHREE (Backend, FastAPI & Integration)

Q1. Why did you choose FastAPI over Flask or Django?

Answer: FastAPI is built on Starlette and Pydantic, offering native asynchronous concurrency (async/await), automatic request validation, high throughput on ASGI servers like Uvicorn, and automatic interactive Swagger documentation at /docs.

Q2. What API endpoints are implemented in DeepGuard?

Answer: POST /api/v1/analyze/image (synchronous image analysis), POST /api/v1/analyze/video (asynchronous video job initialization), GET /api/v1/jobs/{job_id} (job status polling), POST /api/v1/jobs/export/pdf & /json (audit report export), and GET /health (health check).

Q3. Why is video analysis processed asynchronously while images are processed synchronously?

Answer: Single image inference takes ~40 milliseconds, so the HTTP connection can easily wait for a response. Video analysis requires extracting and analyzing dozens of frames across multiple seconds. A synchronous request would freeze the client connection or trigger gateway timeouts.

Q4. How is CORS (Cross-Origin Resource Sharing) handled?

Answer: We configured FastAPI's CORSMiddleware with allow_origins=['*'], allow_methods=['*'], and allow_headers=['*']. This allows the Vite development server running on port 5173 to communicate seamlessly with the FastAPI backend on port 8000.

Q5. What file security validations are performed before running inference?

Answer: In routes_analyze.py, we validate the file extension against a whitelist (.jpg, .png, .webp, .mp4, .mov, .avi), enforce maximum size limits (15MB for images, 100MB for videos), decode bytes with OpenCV to detect corrupt files, and sanitize filenames to prevent path traversal attacks.

Q6. What happens if a corrupt or undecodable image is uploaded?

Answer: OpenCV's cv2.imdecode() returns None. The backend intercepts this immediately and returns an HTTP 400 Bad Request with a clear JSON error message: 'Corrupted or undecodable image file', avoiding unhandled server crashes.

Q7. How does the backend serve both the React Frontend and the API on a single port for production?

Answer: In backend/app/main.py, if the compiled frontend/dist directory exists, we mount /assets via StaticFiles and add a catch-all route that serves frontend/dist/index.html. This means a single Docker container can serve the entire website and API on port 8000.

Q8. How are temporary files managed during video processing?

Answer: Uploaded video streams are written to a secure temporary path using Python's NamedTemporaryFile. As soon as frame extraction and analysis finish (or if an error occurs), a finally block invokes cleanup_file_safely() to delete the file from disk, protecting user privacy.

Q9. How are PDF forensic reports generated?

Answer: We use the ReportLab Python library. In report_generator.py, we construct a structured PDF document containing the file metadata, SHA-256 hash, verdict badge, quality diagnostics, calibrated score, and forensic limitations.

Q10. What is the role of Uvicorn in this project?

Answer: Uvicorn is a lightning-fast ASGI (Asynchronous Server Gateway Interface) web server implementation for Python. It listens on network sockets, translates incoming HTTP requests, and executes FastAPI's asynchronous application code.

Q11. What HTTP status codes does the API return?

Answer: 200 OK for successful analysis, 400 Bad Request for unsupported file types or corrupt media, 422 Unprocessable Entity for schema validation errors, and 500 Internal Server Error for unhandled exceptions.

Q12. How does the feedback feature work in routes_jobs.py?

Answer: Users can submit feedback (e.g., True Positive / False Positive report) via POST /api/v1/jobs/feedback. The server logs the feedback for manual inspection without automatically retraining the model on unverified data.

SECTION D: VIVA QUESTIONS FOR KRUSHI (Preprocessing, Datasets & Quality)

Q1. Which public datasets are standard for deepfake detection research?

Answer: FaceForensics++ (FF++) contains 1,000 original YouTube videos manipulated with 4 methods (Deepfakes, Face2Face, FaceSwap, NeuralTextures). Celeb-DF v2 has 5,639 high-quality celebrity deepfake videos. DiffusionDB provides 14M images generated by Stable Diffusion.

Q2. Why is face detection and extraction necessary instead of passing the whole photo to the model?

Answer: Face-swapping techniques only alter the facial region. The background (walls, trees, clothing) remains 100% authentic camera data. If you pass the entire image, background pixels dominate the convolution operations and dilute the subtle manipulation artifacts on the face.

Q3. How does our face extractor detect human faces?

Answer: We use OpenCV's Haar Feature-based Cascade Classifiers (haarcascade_frontalface_default.xml). It calculates edge and line features using integral images across different scales to detect frontal facial regions rapidly.

Q4. What is the purpose of Non-Maximum Suppression (NMS) in face extraction?

Answer: Multi-scale detection often identifies multiple overlapping bounding boxes around the same person's face. Non-Maximum Suppression compares the overlap (Intersection-over-Union, IoU) and suppresses redundant boxes, keeping only the single highest-confidence bounding box per subject.

Q5. Why is 25% Margin Padding added around the face bounding box?

Answer: Most face-swapping algorithms paste a synthesized mask onto a target face. The most tell-tale artifacts (blending seams, color tone discontinuities, and blurring) occur along the jawline, chin, and hair borders. Tight face crops discard these borders. Adding a 25% margin ensures the CNN inspects the transition boundaries.

Q6. How is Motion Blur detected mathematically?

Answer: We convert the image to grayscale and compute the Variance of the Laplacian (sigma^2 = Var(del^2 I)). The Laplacian operator calculates 2nd-order spatial derivatives, highlighting rapid intensity changes (sharp edges). A sharp image has high variance; a blurred image has low variance (<50.0).

Q7. How do we test for poor lighting or low contrast?

Answer: We compute the standard deviation of pixel intensities across the grayscale channel. Low standard deviation (<20.0) indicates that all pixels cluster in a narrow luminance range (very dark or flatly lit), which prevents the model from observing fine facial pores.

Q8. What image normalization is performed before PyTorch inference?

Answer: We resize the cropped face to 224x224 pixels, convert color channels from BGR to RGB, scale pixel values to [0.0, 1.0], and normalize using standard ImageNet mean ([0.485, 0.456, 0.406]) and standard deviation ([0.229, 0.224, 0.225]).

Q9. How are video frames sampled in video_processor.py?

Answer: We read the video's frames-per-second (FPS) metadata using OpenCV. We calculate a step interval to extract exactly 1 frame per second, up to a maximum cap of 30 or 60 frames. This provides temporal coverage across the entire duration without loading hundreds of redundant frames.

Q10. What happens if an image or video contains no detectable human faces?

Answer: Our pipeline detects len(faces) == 0 and stops before running CNN inference. It returns an honest verdict: 'Unable to Analyze: No human face detected'. It never invents a random prediction or assumes that a blank image is real.

Q11. What is the difference between BGR and RGB color formats?

Answer: OpenCV decodes images into BGR (Blue-Green-Red) by default, while PyTorch and PIL expect RGB (Red-Green-Blue). Failing to convert with cv2.cvtColor(img, cv2.COLOR_BGR2RGB) inverts the red and blue channels, drastically lowering detection accuracy.

Q12. What is the difference between Facial Manipulation mode and AI-Generated mode in preprocessing?

Answer: Facial Manipulation mode isolates, crops, and pads human face bounding boxes to inspect seam artifacts. AI-Generated Synthetic mode evaluates the global, uncropped image because diffusion models synthesize the entire scene (landscapes, lighting, backgrounds).

■ SECTION E: VIVA QUESTIONS FOR SUSHTI (Explainability, Testing & Scope)

Q1. What is Grad-CAM, and how does it work?

Answer: Grad-CAM stands for Gradient-weighted Class Activation Mapping. It takes the gradient of the predicted deepfake class score with respect to the feature activation maps of the final convolutional layer (features[8]). By performing global average pooling on these gradients, it calculates importance weights for each feature map, sums them, and passes them through a ReLU function to generate a coarse 2D heatmap showing where the model focused its attention.

Q2. Why is ReLU applied at the end of Grad-CAM?

Answer: We only care about features that have a positive influence on the manipulation score (features that increase the likelihood of the image being a deepfake). Negative gradients correspond to features that suggest the image is authentic; ReLU discards negative values and keeps only positive evidence.

Q3. Does a Grad-CAM heatmap prove legally that a specific pixel was edited?

Answer: No. Grad-CAM shows neural network attention, not ground truth forensic logging. It proves that the model based its decision on artifacts in that region (e.g., around the mouth), serving as supporting forensic evidence rather than definitive legal proof.

Q4. How does the 5-Anatomical Zone breakdown work?

Answer: We divide the normalized face coordinate bounding box into 5 geometric sectors: Forehead, Eyes, Nose, Mouth, and Jawline/Cheeks. We calculate the mean Grad-CAM activation density within each sector's bounding polygon to give an individual risk rating (e.g., Mouth: 82% Risk, Eyes: 12% Risk).

Q5. How does video score aggregation work (Top-k vs Average)?

Answer: If a 30-second video has a 2-second deepfake insertion, simple averaging across all 30 frames would dilute the fake score, resulting in a false negative. DeepGuard uses weighted top-k pooling: 70% weight is given to the highest suspicious frame score, and 30% to the overall mean. This catches brief, localized manipulations.

Q6. What is the Temporal Persistence rule in video analysis?

Answer: Single-frame anomalies can occur due to quick head turns or camera motion blur. Temporal persistence checks whether elevated manipulation scores appear across consecutive or clustered frames rather than an isolated outlier frame, boosting diagnostic confidence.

Q7. What are the known limitations of DeepGuard?

Answer: 1. Heavy Social Media Compression: Platforms like WhatsApp aggressively downscale and re-compress images (JPEG blockiness), smoothing out microscopic synthetic artifacts. 2. Audio Deepfakes: DeepGuard currently evaluates visual frames; it does not analyze cloned audio or voice dubbing. 3. Novel Diffusion Models: As new generative architectures emerge, unseen artifact patterns may score near the inconclusive zone until fine-tuned.

Q8. What happens if multiple faces are present in a photo?

Answer: The Haar Cascade extracts all valid faces, pads each with a 25% margin, and evaluates them independently. The overall image takes the verdict of the highest-risk face, and Poonam's frontend displays the interactive FaceSelector so the investigator can click and inspect each person individually.

Q9. How did you test system integration across different devices?

Answer: We tested across three dimensions: 1. Input Types: Uploaded JPG/PNGs, MP4/MOV videos, and live WebRTC camera snapshots. 2. Stress Conditions: Deliberately blurry photos, low-light selfies, and multi-face portraits. 3. Responsive Viewports: Mobile screens, laptops, and dark/light themes.

Q10. What are the future enhancements planned for DeepGuard?

Answer: Integrating audio-visual sync analysis (detecting lip movement mismatch with speech); adding a Frequency Domain branch (Spatial-Frequency dual-stream CNN using Discrete Fourier Transform); and deploying a mobile app using React Native or Flutter.

Q11. How does DeepGuard protect user privacy during analysis?

Answer: Media files are processed locally in server memory and temporary video files are deleted immediately after keyframe extraction. Uploaded photos are never retained or added to training databases without explicit consent.

Q12. What advice should students give when an examiner asks about unsupported features?

Answer: Be transparent and honest! If asked about audio deepfakes, reply: 'Our current model focuses on spatial visual artifacts. Audio deepfake analysis is our proposed future enhancement.' Examiners respect precise, honest engineering answers.

PART 4: EXAMINER EVALUATION MATRIX

Member	Module Focus	Key Code Files	Demonstration Task in Viva
Varsha	CNN & Inference	model_loader.py, config.py	Explain compound scaling, dual thresholds (0.35/0.55), accuracy
Poonam	Frontend UI/UX	App.jsx, Analyze.jsx, LiveCameraCapture.jsx	Live camera capture & track cleanup, 3D face mesh pause, them
Anushree	Backend & APIs	main.py, routes_analyze.py, routes_jobs.py	Swagger docs at /docs, async video polling, file validation, securi
Krushi	Preprocessing & Data	face_extractor.py, video_processor.py	Haar Cascades, 25% margin padding, Laplacian blur check, 1-FP
Sushti	Explainability & Tests	HeatmapViewer.jsx, VideoTimeline.jsx	Wipe before/after split slider, Grad-CAM layer features[8], limitatio